

**Московский государственный технический
университет им. Н.Э. Баумана**

**Факультет информатика и системы управления
Кафедра системы обработки информации и управления**

Курс «Парадигмы и конструкции языков программирования»

Отчет по рубежному контролю №2

Вариант Б12

Выполнил:
студент группы ИУ5-32Б:

Пчелинцева Д.А.
Подпись и дата:
21.12.2025г.

Проверил:
преподаватель каф.
ИУ5

Гапанюк Ю.Е.
Подпись и дата:

Москва, 2025 г.

Задание:

Условия рубежного контроля №2 по курсу ПИК ЯП

Рубежный контроль представляет собой разработку тестов на языке Python.

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Файл rk2_refactored.py:

```
from operator import itemgetter

class Lang:
    """ Язык программирования """
    def __init__(self, id, name, year, app_id):
        self.id = id
        self.name = name
        self.year = year
        self.app_id = app_id

class App:
    """ Среда разработки """
    def __init__(self, id, name):
        self.id = id
        self.name = name

class LangApp:
    """ Реализация многие-ко-многим """
    def __init__(self, lang_id, app_id):
        self.lang_id = lang_id
        self.app_id = app_id

def get_apps():
    """ Возвращает список объектов App """
    return [
        App(1, "Visual Studio"),
        App(2, "PyCharm"),
        App(3, "Eclipse"),
        App(4, "VS Code"),
        App(5, "IntelliJ IDEA")
    ]

def get_langs():
    """ Возвращает список объектов Lang """
    return [
        Lang(1, "Python", 1991, 2),
        Lang(2, "Java", 1995, 3),
        Lang(3, "C#", 2000, 1),
        Lang(4, "JavaScript", 1995, 4),
```

```

        Lang(5, "Kotlin", 2011, 5),
        Lang(6, "TypeScript", 2012, 4),
        Lang(7, "Паскальов", 1970, 3) # добавлен для БЗ
    ]

def get_lang_apps():
    """ Возвращает список объектов LangApp """
    return [
        LangApp(1, 2),
        LangApp(1, 4),
        LangApp(2, 3),
        LangApp(2, 5),
        LangApp(3, 1),
        LangApp(4, 4),
        LangApp(5, 5),
        LangApp(6, 4),
        LangApp(6, 2),
        LangApp(7, 5)
    ]

def create_one_to_many(langs, apps):
    """ Создает список связей один-ко-многим (язык -> среда) """
    return [(l.name, l.year, a.name)
            for l in langs
            for a in apps
            if l.app_id == a.id]

def create_many_to_many(langs, apps, lang_apps):
    """ Создает список связей многие-ко-многим (язык -> среда) """
    many_to_many_temp = [(a.name, la.app_id, la.lang_id)
                          for a in apps
                          for la in lang_apps
                          if a.id == la.app_id]
    return [(l.name, l.year, app_name)
            for app_name, app_id, lang_id in many_to_many_temp
            for l in langs if l.id == lang_id]

def task_b1(one_to_many_data):
    """ Задача Б1: Список всех связанных языков и сред, отсортированный по языкам """
    return sorted(one_to_many_data, key=itemgetter(0))

def task_b2(apps, one_to_many_data):
    """ Задача Б2: Список сред с количеством языков, отсортированный по количеству """
    res_unsorted = []
    for a in apps:
        langs_in_app = list(filter(lambda x: x[2] == a.name, one_to_many_data))
        count = len(langs_in_app)
        if count > 0:
            res_unsorted.append((a.name, count))

```

```

        return sorted(res_unsorted, key=itemgetter(1))

def task_b3(many_to_many_data):
    """ Задача Б3: Список языков, заканчивающихся на 'ов', и их сред """
    res_b3 = [
        (lang_name, app_name)
        for lang_name, year, app_name in many_to_many_data
        if lang_name.endswith("ов")
    ]
    res_b3_unique = list(set(res_b3))
    return sorted(res_b3_unique) if res_b3_unique else []

def main():
    """ Основная функция, которая вызывает все задачи и выводит результаты """
    apps = get_apps()
    langs = get_langs()
    lang_apps = get_lang_apps()

    one_to_many = create_one_to_many(langs, apps)
    many_to_many = create_many_to_many(langs, apps, lang_apps)

    print('Task Б1')
    res_1 = task_b1(one_to_many)
    print(res_1)

    print('\nTask Б2')
    res_2 = task_b2(apps, one_to_many)
    for app, cnt in res_2:
        print(f"Среда разработки: {app}, Кол-во языков: {cnt}")

    print("\nTask Б3")
    res_b3 = task_b3(many_to_many)
    if res_b3:
        for lang, app in res_b3:
            print(f"Язык: {lang}, Среда: {app}")
    else:
        print("Нет языков, название которых заканчивается на 'ов'.")

if __name__ == "__main__":
    main()

```

Файл test_rk2.py:

```

import unittest
from rk2_refactored import (
    Lang, App, LangApp,
    create_one_to_many, create_many_to_many,
    task_b1, task_b2, task_b3
)

```

```

class TestRk2(unittest.TestCase):

    def setUp(self):
        """ Подготовка тестовых данных """
        self.apps = [
            App(1, "Visual Studio"),
            App(2, "PyCharm"),
            App(3, "Eclipse"),
            App(4, "VS Code"),
            App(5, "IntelliJ IDEA")
        ]
        self.langs = [
            Lang(1, "Python", 1991, 2),
            Lang(2, "Java", 1995, 3),
            Lang(3, "C#", 2000, 1),
            Lang(4, "JavaScript", 1995, 4),
            Lang(5, "Kotlin", 2011, 5),
            Lang(6, "TypeScript", 2012, 4),
            Lang(7, "Паскальов", 1970, 3)
        ]
        self.lang_apps = [
            LangApp(1, 2),
            LangApp(1, 4),
            LangApp(2, 3),
            LangApp(2, 5),
            LangApp(3, 1),
            LangApp(4, 4),
            LangApp(5, 5),
            LangApp(6, 4),
            LangApp(6, 2),
            LangApp(7, 5)
        ]

    def test_task_b1(self):
        """ Тест для задачи Б1: проверка сортировки по названию языка """
        one_to_many = create_one_to_many(self.langs, self.apps)
        result = task_b1(one_to_many)

        # Проверяем, что список отсортирован по названию языка (первый элемент
кортежа)
        lang_names = [item[0] for item in result]
        self.assertEqual(lang_names, sorted(lang_names))

    def test_task_b2(self):
        """ Тест для задачи Б2: проверка подсчета языков в средах и сортировки по
количеству """
        one_to_many = create_one_to_many(self.langs, self.apps)
        result = task_b2(self.apps, one_to_many)

        # Проверяем, что результат отсортирован по количеству (второй элемент
кортежа)

```

```

counts = [cnt for _, cnt in result]
self.assertEqual(counts, sorted(counts))

expected_counts = {
    "Visual Studio": 1,
    "PyCharm": 1,
    "Eclipse": 2,
    "VS Code": 2,
    "IntelliJ IDEA": 1
}
for app_name, count in result:
    self.assertEqual(count, expected_counts[app_name])

def test_task_b3(self):
    """ Тест для задачи Б3: проверка фильтрации языков, заканчивающихся на
    'ов' """
    many_to_many = create_many_to_many(self.langs, self.apps, self.lang_apps)
    result = task_b3(many_to_many)

    # Проверяем, что в результате есть только язык, заканчивающийся на 'ов'
    self.assertEqual(len(result), 1)
    self.assertEqual(result[0][0], "Паскальов")

    self.assertEqual(result[0][1], "IntelliJ IDEA")

if __name__ == '__main__':
    unittest.main()

```

Результат выполнения:

```

● PS C:\Users\Дарья\OneDrive\VisualStudioCode\Labs_pikep\rk2> python .\test_rk2.py
...
-----
Ran 3 tests in 0.001s

OK

```